

TECHCATALYST DE 2026

Data architectures and pipeline design

Turn business needs into specs, learn the patterns, then design your own pipeline and defend it.

The Hartford · TechCatalyst Data Engineering 2026

Week 1 Day 5

Requirements to specs

Lambda vs Kappa

Medallion architecture

Diagramming with Draw.io

Read and design pipelines

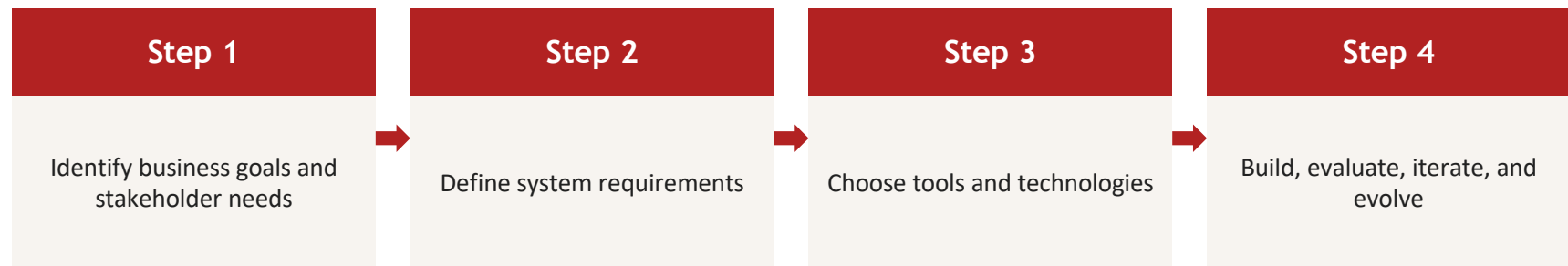
DAY 5

Today's agenda

Time	Block
8:00 to 8:30	Recap and stand-up
8:30 to 9:25	Think like an architect: requirements → spec, DE lifecycle
9:25 to 10:00	Pipeline vocabulary, ETL / ELT / Reverse ETL
10:00 to 10:15	Break
10:15 to 10:50	Patterns: batch/streaming, Lambda, Kappa, Medallion, orchestration
10:50 to 11:30	Lab A: read a reference architecture
11:30 to 12:00	Diagram grammar, Draw.io demo, meet the dataset
1:00 to 3:00	Lab B: design your taxi pipeline (the deliverable)
3:00 to 3:15	Break
3:15 to 4:15	Readouts and critique
4:15 to 5:00	Week 1 synthesis, recap, exit ticket

PROCESS

Thinking like a data engineer



The process is not linear: you loop back as you learn. Requirements change and designs evolve; step 1 (who you serve and what they will do with the data) anchors all of it.

Today is steps 1 and 2: understand the need, define the requirements. Steps 3 and 4 are the rest of the program.

Architecture is decisions, not boxes

“Good architecture answers questions before they are asked.”

Question	Drives
What are the sources, formats, and volumes?	Ingestion pattern, storage layout
How fresh must the data be?	Batch vs streaming, and cost
Who consumes it, and how?	Warehouse design, serving layer, BI
What can go wrong?	Quality checks, retries, monitoring
What is sensitive?	PII masking, access control, governance
What does it cost to run?	Service choices, partitioning, lifecycle

REQUIREMENTS

Three levels of requirements

Business requirements: high-level goals of the business (grow revenue, reduce fraud losses, improve retention).

Stakeholder requirements: what individuals need to do their job (an analyst needs daily zone-level trip counts).

System requirements

Functional (the WHAT the system must do) + Non-functional (the HOW WELL it must do it).

Business and stakeholder needs are the “why.” System requirements are the “what” and “how well.” A data engineer translates down the stack.

REQUIREMENTS

Functional vs non-functional requirements

Functional

the WHAT it must do

- Data sources
- Data storage
- Data querying
- Visualization
- Stakeholders and consumers
- Real-time vs batch

Non-functional

the HOW WELL it must do it

- Scalability
- Performance
- Reliability
- Availability
- Security
- Observability

Beginners specify only functional needs. Non-functional requirements (scale, reliability, security, cost) are where real designs are won or lost.

CORE DE SKILL

From business need to data spec

Stakeholder: “Tell me which neighborhoods need more cars at 5pm on weekdays.”

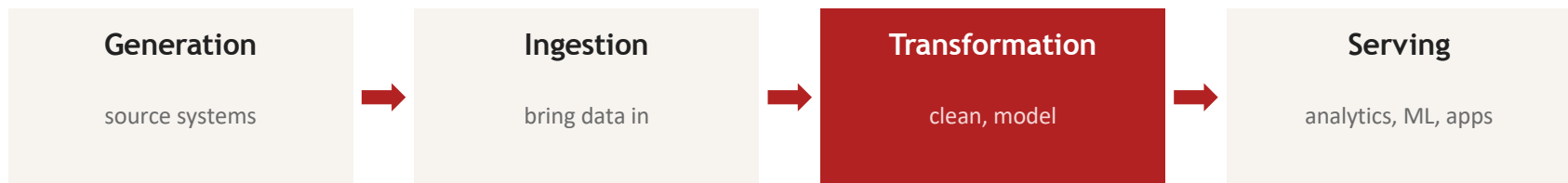
The data engineer translates that into a spec:

Spec field	Value for this ask
Sources	TLC yellow trips (pickup zone, datetime) + taxi zone lookup
Grain	Trips per pickup zone, per hour
Freshness	Daily refresh is enough; not real-time
Metrics	Pickups by zone and hour, weekday 5pm window, week over week
Sensitive	No direct PII, but zone plus time can re-identify: aggregate
Cost driver	Full scans of years of trips: partition by date

A spec is the bridge: vague business language on one side; concrete sources, grain, freshness, and SLAs on the other.

FRAMEWORK

The data engineering lifecycle



Undercurrents (run under every stage):

Security · Data management · DataOps · Orchestration · Software engineering

Stages move data left to right; the undercurrents are the disciplines that make it trustworthy. Today's patterns and diagrams all live inside this frame.

PIPELINE VOCABULARY

What is a data pipeline?

Definition

A data pipeline is a repeatable path that moves data from where it is generated to where it becomes useful.

Move

Extract or receive data from files, APIs, apps, streams, or databases.

Check

Validate schema, quality, security, freshness, and completeness.

Shape

Clean, standardize, join, aggregate, enrich, and model.

Serve

Publish data to a warehouse, lakehouse table, dashboard, app, model, or API.

PIPELINE VOCABULARY

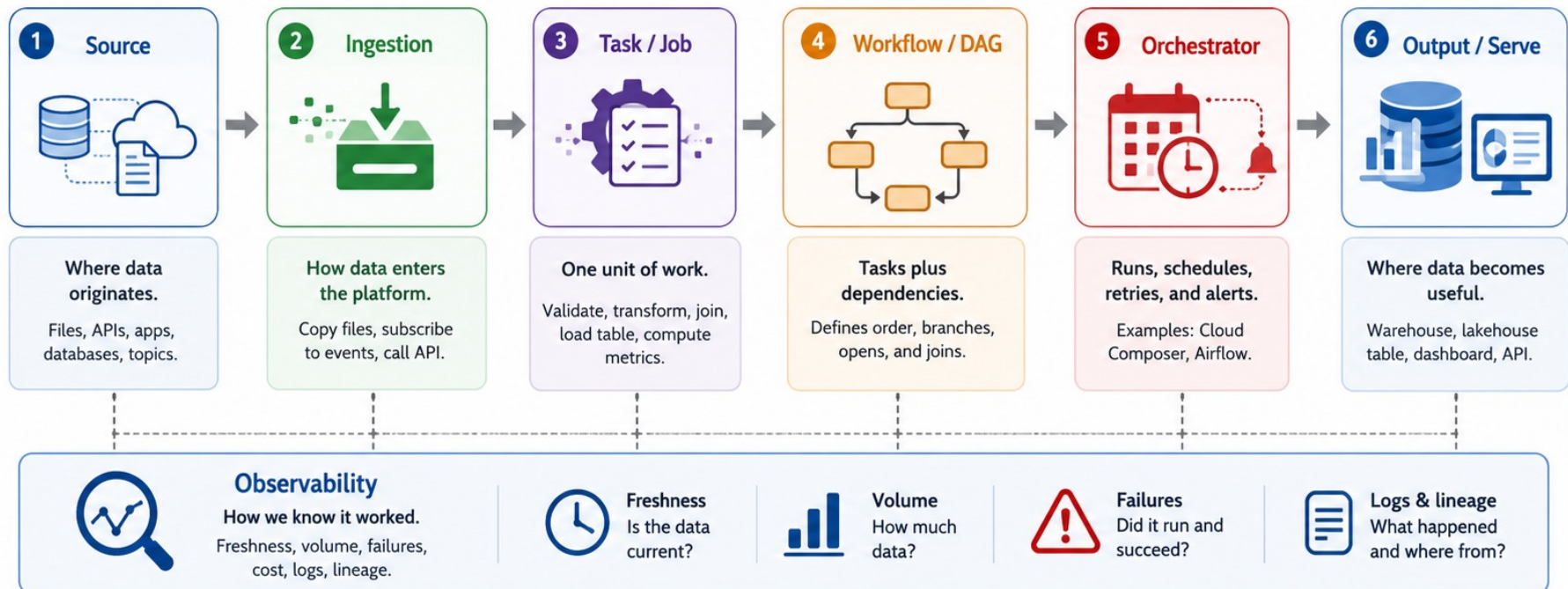
Pipeline components at a glance

	Source Where data originates.	The systems and locations that generate raw data.	Examples  TLC Parquet file  API  App database  Pub/Sub topic
	Ingestion How data enters the platform.	The mechanisms used to bring data into your environment.	Examples  Copy file to GCS  Subscribe to events  Call API
	Task / job One unit of work.	A single step that performs a specific operation on data.	Examples  Validate file  Transform trips  Load BigQuery table
	Workflow / DAG Tasks plus dependencies.	A directed graph that defines the order in which tasks run.	Examples  Load zones →  Join trips →  Write to zones
	Orchestrator Runs, schedules, retries, alerts.	The system that executes workflows and manages their lifecycle.	Examples  Cloud Composer  Apache Airflow
	Observability How we know it worked.	The signals and logs that help us monitor health, quality, and performance.	Examples  Freshness  Volume  Failures  Cost  Logs

PIPELINE VOCABULARY

Pipeline components: what each piece does

A pipeline is made of connected components that move data from source to value.



PIPELINE VOCABULARY

Pipeline pieces: tasks, workflows, and orchestrators

Term	Meaning	Example
Source	Where data originates	TLC Parquet file, API, app database, Pub/Sub topic
Ingestion	How data enters the platform	Copy file to GCS, subscribe to events, call API
Task / job	One unit of work	Validate file, transform trips, load BigQuery table
Workflow / DAG	Tasks plus dependencies	Load zones before joining trips to zones
Orchestrator	Runs, schedules, retries, alerts	Cloud Composer / Airflow
Observability	How we know it worked	Freshness, volume, failures, cost, logs

DATA PIPELINES

ETL vs ELT vs Reverse ETL

ETL

Extract data, transform it first, then load curated data into the destination.



Best when data must be cleaned and shaped before loading.



ELT

Extract data, load raw data first, then transform it inside the warehouse or lakehouse.



Best for modern cloud platforms, scale, and keeping raw history.



Reverse ETL

Take trusted data from the warehouse/lakehouse and sync it back into business tools.



Best for operationalizing analytics in day-to-day business tools.



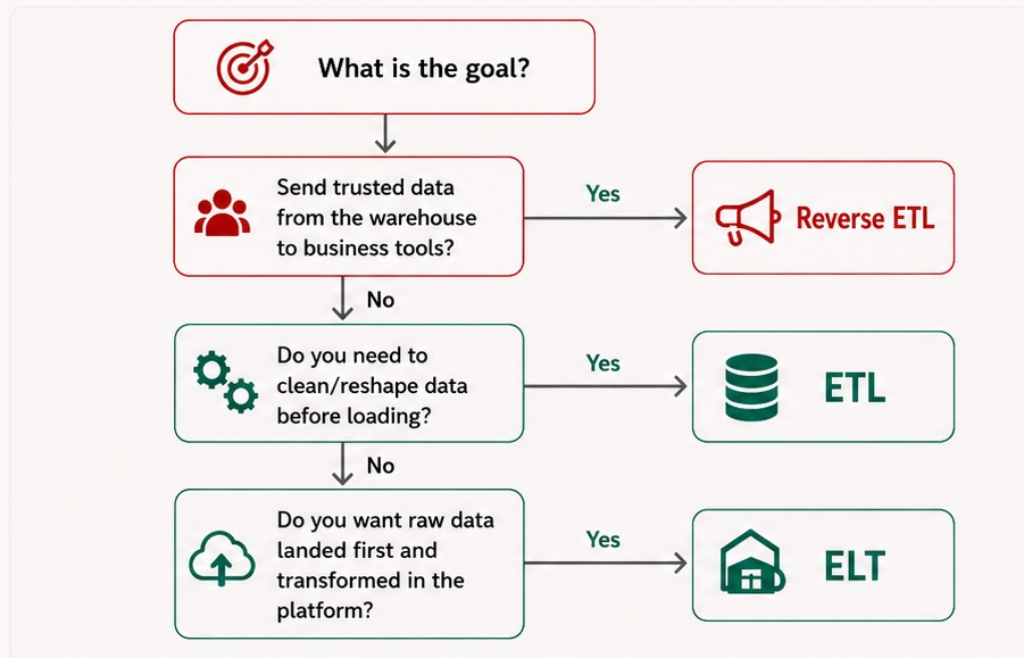
Think of the difference as **WHERE** transformation happens and **WHERE** the final data needs to go.



Teaching note: Reverse ETL is not the opposite of ETL; it activates analytics-ready data.

DESIGN DECISIONS

How to choose: ETL, ELT, or Reverse ETL



ETL

- Transform before load
- Good for strict schemas, legacy systems, or pre-load quality checks
- Useful when the destination should only receive curated data



ELT

- Load first, transform later
- Great for cloud warehouses/lakehouses, scale, and raw data retention
- Useful for iterative analytics, SQL/dbt workflows, and ML



Reverse ETL

- Activate data outward
- Sync warehouse metrics, segments, or scores into operational tools
- Useful for CRM, marketing automation, support, and sales workflows



Quick rule of thumb



ETL = transform before destination



ELT = transform inside the destination



Reverse ETL = send trusted data back out to the business

ETL vs ELT: where transformation happens

ETL: Extract → Transform → Load

Transform before loading the final target. Useful when the destination is constrained, sensitive data must be shaped before arrival, or legacy warehouse loading rules require it.

ELT: Extract → Load → Transform

Load raw or lightly processed data first, then transform inside the warehouse/lakehouse. Common with BigQuery, Snowflake, dbt, and medallion-style refinement.

Modern default for analytics is often ELT, but ETL still matters when governance, latency, data contracts, or target limits require earlier shaping.

Reverse ETL: analytics back into operations

What it is

Reverse ETL moves trusted analytical outputs from the warehouse/lakehouse back into operational systems where people and applications act on them.

Examples

Churn score to CRM
Customer segment to marketing tool
Risk flag to internal operations app

Why teams use it

Insights are only valuable when they reach the workflow where decisions happen.

Risks

PII leakage, stale scores, unclear ownership, accidental feedback loops.

PIPELINE VOCABULARY

Pipeline vs orchestration vs architecture

Concept	Answers	What students should draw or label
Pipeline	What happens to the data?	Source → ingest → store → transform → serve
Orchestration	How does the work run reliably?	Schedule, dependencies, retries, alerts, backfills
Architecture	Why are these components arranged this way?	Requirements, trade-offs, service choices, boundaries
Governance	Who can access what, and under what rules?	PII controls, IAM, catalog, retention
Observability	How do we know it worked?	Freshness, row counts, failures, cost, data quality checks

In Lab B, your diagram must show all five: flow, run behavior, design rationale, governance, and observability.

01

Architecture patterns

Three names you will hear in every design review.

Batch vs streaming: the first decision

Batch

- Process data in scheduled chunks (hourly, nightly)
- Simpler and cheaper; easy to reprocess
- Higher latency: results are minutes to hours old
- Most enterprise data, including insurance, is batch

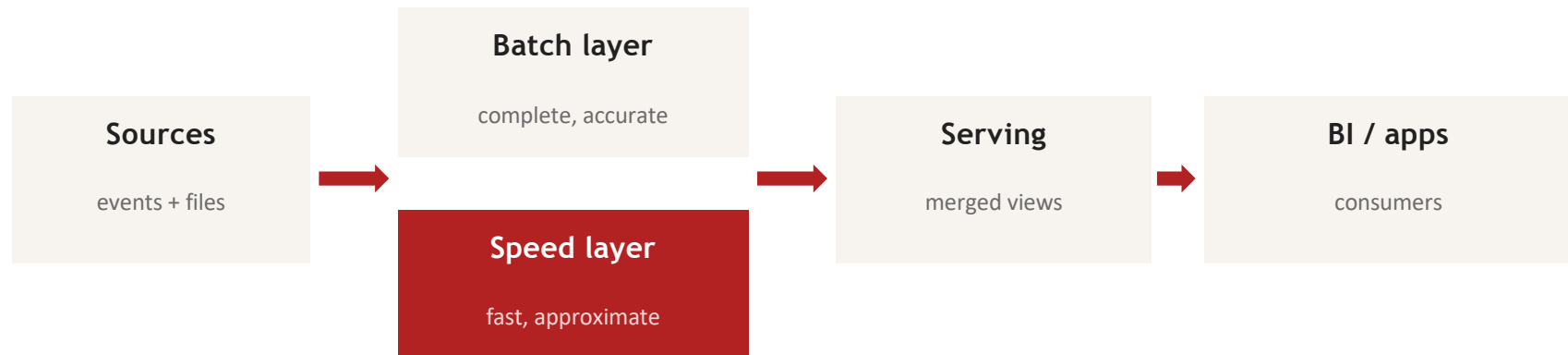
Streaming

- Process events continuously as they arrive
- Low latency: results in seconds
- More complex and costly to run and reason about
- Use when freshness drives value (fraud, telematics)

The question is not “which is better” but “how fresh must this consumer's data be, and what will that freshness cost?”

PATTERNS

Lambda architecture: two paths, one truth



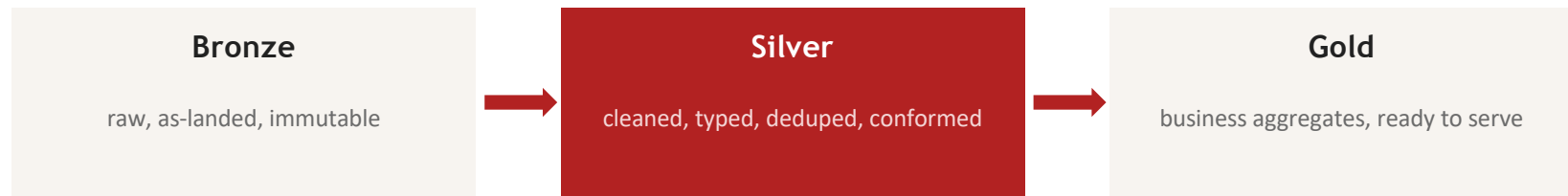
Pro: accuracy and low latency together. Con: two codebases doing the same logic, expensive to build and keep in sync.

Kappa architecture: everything is a stream



- One processing path: reprocess history by replaying the log from the start
- Simpler than Lambda, but everything must fit the streaming model
- Reality check: most enterprises (including insurers) run mostly-batch hybrids, and that is fine

Medallion architecture: refine in layers



- Each layer has a contract: what is guaranteed about the data once it is there
- Never edit bronze; you can always rebuild silver and gold from it
- Maps directly to our buckets: raw/ to cleaned/ to curated/. You will live this from Week 2 on

PATTERNS

ETL vs ELT (a Week 3 preview)

ETL

- Transform before loading
- Pattern of the on-prem era: warehouse compute was scarce
- Extract, then Transform, then Load

ELT

- Load raw, transform inside the warehouse
- Modern default: BigQuery and Snowflake compute is elastic
- Extract, then Load, then Transform (dbt)

Medallion is ELT thinking applied to the lake: land first, refine in place. Full treatment in Week 3.

SYNTHESIS

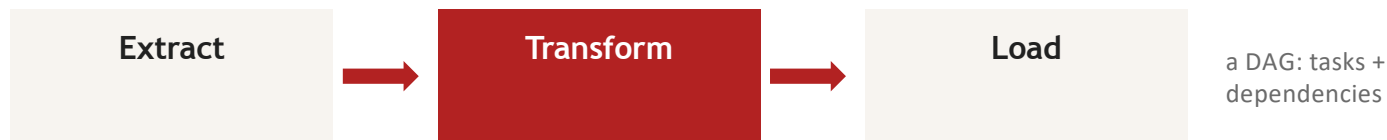
Which pattern when

Pattern	Reach for it when	Watch out for
Batch	Most analytics; freshness in hours is fine	Latency; nightly windows
Streaming	Freshness in seconds creates value	Complexity and cost
Lambda	You need accurate and fast, and can afford two paths	Duplicated logic to maintain
Kappa	Sources are naturally events; you want one codebase	Everything must fit streaming
Medallion	Almost always, to organize lake/warehouse refinement	It is layering, not a speed choice

Medallion is how you **organize**; batch / streaming / Lambda / Kappa is **how fresh**. Most teams: batch + medallion, streaming only where it pays.

OPERATING THE PIPELINE

Orchestration: what makes the pipeline run



- An orchestrator runs tasks in dependency order, on a schedule, and retries failures
- Schedule: cron-like (nightly at 2am) or event-driven (a file lands)
- Retries and alerts: a failed task retries, then pages someone; idempotent tasks make retries safe
- Backfills: rerun the DAG for past dates when logic changes or data arrives late

Tool: Cloud Composer (managed Apache Airflow) on GCP; built in Week 3. Today: know the words, and mark schedules on your diagram.

Data quality and observability: design for failure

- Pipelines fail constantly: late files, schema drift, nulls, duplicates, \$500k fare outliers. Design for failure
- Place quality gates at zone boundaries: validate into bronze (did the file arrive, right schema?) and into silver (types, ranges, dedupe)
- Quarantine bad data instead of dropping it: a rejected-records path keeps a trace you can investigate
- Make pipelines idempotent and replayable: same input gives same output; immutable bronze lets you rebuild
- Observability: watch freshness, volume, and failures before a consumer tells you

On a diagram, a quality check is a ⚠️ at a zone boundary. Decide where it lives before you draw the box.

GOVERNANCE

Data contracts and schema evolution

- A data contract is an agreement about a dataset's shape: columns, types, semantics, and guarantees between a producer and its consumers
- Schemas drift: the TLC added and renamed columns across years; a contract makes drift explicit instead of silently breaking downstream
- Evolve deliberately: additive changes (a new nullable column) are safe; renames and type changes are breaking and need versioning
- Enforce at zone boundaries: validate incoming data against the contract before it enters silver

A contract turns “the file changed and everything broke” into “the producer changed the contract; here is the new version.”

02

Drawing architectures

A diagram is an argument: make it readable.

DIAGRAMMING

Diagram grammar

Rule	Why
Flow one direction (left to right)	Readers should not hunt for the start
Label every arrow with what moves	“parquet files, hourly” beats a bare line
Group into zones (ingest / store / transform / serve)	Layers tell the story at a glance
One icon = one service, named	“Dataflow,” not “magic happens”
Show the humans and consumers	Architecture exists for them
Note latency, volume, schedule	The numbers drive the design

LEARN FROM THE PROS

Reference architectures: GCP and AWS

- Cloud providers publish vetted reference architectures: proven designs for common problems
- Read them to learn the grammar before you design; borrow patterns, do not copy blindly
- How to read one: find the zones, follow the data flow, name the services, ask where quality, PII, and cost live

GCP Cloud Architecture Center

cloud.google.com/architecture

Browse the Data analytics section

AWS Architecture Center

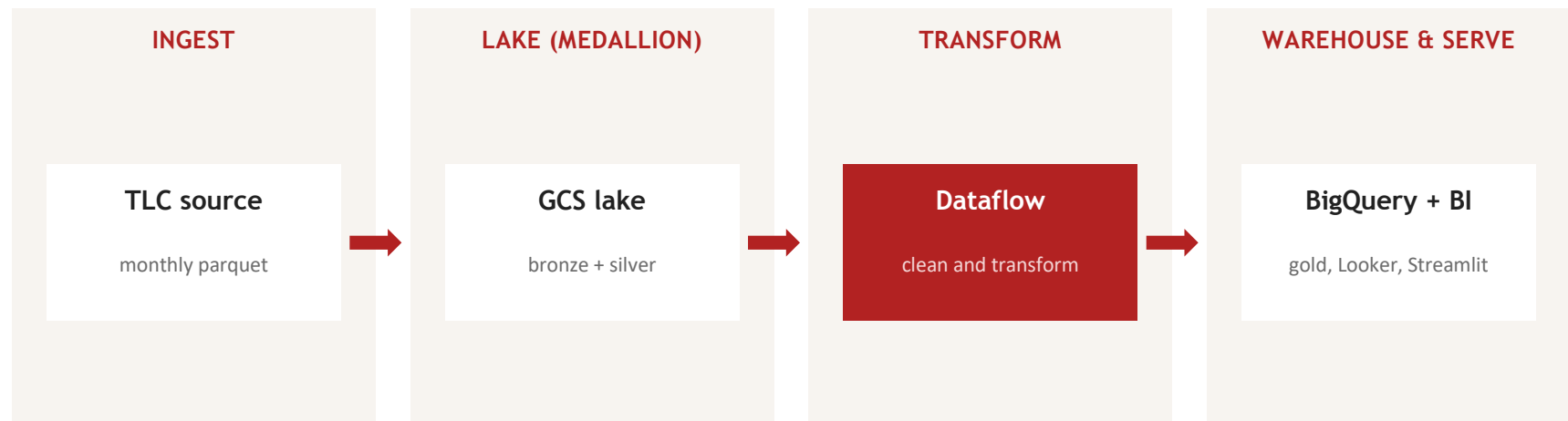
aws.amazon.com/architecture

Analytics reference architecture diagrams

Lab A: you analyze one of these and compare it to the medallion model before you design your own.

DIAGRAMMING

Reference: NYC Taxi pipeline on GCP



This is the skeleton you will flesh out over 8 weeks. Today you design your own version of it.

PROJECT THREAD

Meet the dataset: NYC TLC trip records

3M+

yellow taxi trips per month, since 2009

Parquet

columnar files published monthly by NYC TLC

4

trip types: yellow, green, FHV, high-volume FHV

- Plus lookup tables (taxi zones), shapefiles, and related NYC open data (collisions, air quality)
- Real production messiness: nulls, outliers (\$500k fares), schema drift across years, late data
- The same dataset used by the DataTalksClub DE Zoomcamp, a strong free companion reference

LAB A

Read a reference architecture

Goal: learn the grammar by analyzing a real published architecture before you design your own.

40 min · pairs

- Pick one data-analytics reference architecture from the GCP or AWS architecture center
- Annotate it: zones, data flow, and the named services in each zone
- Mark where quality checks, PII handling, and the top cost driver live
- Map it onto the medallion model and note one idea you will reuse in Lab B

Deliverable: the annotated diagram (or notes) plus your medallion mapping, in your team folder.

LAB B

Design your taxi pipeline (deliverable)

Goal: produce your team's end-to-end conceptual architecture for the 8-week pipeline.

120 min · teams

- Translate the business ask into a data spec, then answer the six architect questions (no drawing yet)
- Draw the flow in Draw.io: sources to ingest to medallion zones to warehouse to serving
- Label arrows with data, format, and cadence; mark ⚠ quality checks and 🔒 PII handling
- Name real services (GCP primary; note AWS equivalents); reuse yesterday's storage layout

Deliverable: data spec + architecture.drawio + architecture.png + design_narrative.md. Week 1's graded deliverable.

Critique protocol for readouts

Presenters

- Start with the spec and requirements, not boxes
- Walk left to right, zone by zone
- Name every service and why it is there
- Admit unknowns; that is a strength

Reviewers ask

- What happens when a file fails halfway?
- Where would PII leak?
- What does this cost at 10x volume?
- What would you cut to ship in a week?

Critique the design, never the designer. Every architecture is a draft.

WEEK 1 IN ONE PICTURE

Five days, one arc



From “what is data engineering” to “here is the pipeline I will build.” Next week you start building it.

THE ROAD AHEAD

Your pipeline over 8 weeks

- 1 Done: cloud, dev environment, storage, and your architecture blueprint
- 2 Linux, Git collaboration, Python from scratch (still AI-free)
- 3 BigQuery and SQL: ingest and warehouse layers
- 4 Snowflake, dbt, advanced SQL: the transform layer
- 5 PySpark and big data: scale the processing
- 6 GenAI and AI-assisted engineering (Copilot returns)
- 7 BI and data apps: the serving layer
- 8 Capstone: your blueprint becomes a working pipeline

The diagram you draw today is the backbone of all of it.

RECAP

Key takeaways

1

The core DE skill: translate business needs into a data spec, then a defended design

2

Architecture starts with questions (freshness, volume, consumers, sensitivity, cost), not services

3

Lambda = two paths; Kappa = one stream; Medallion = refine raw to silver to gold; orchestration runs it

4

Design for failure: quality gates, quarantine, replayability, and data contracts

5

Your Week 1 deliverable is the blueprint you will actually build for 7 more weeks

EXIT TICKET

Wrap the week

1

Take a business ask and translate it into a data spec: name the sources, grain, freshness, and one sensitive field.

2

In one sentence each, contrast Lambda, Kappa, and Medallion.

3

Where on your diagram does a malformed file get caught, and where does it go?

Before you leave: commit your data spec, architecture.drawio, architecture.png, and design_narrative.md.